



# Konsep *Collection*

IF2210 – Semester II 2025/2026

SAR

# Collection

- Di dunia nyata maupun dalam pemrograman, seringkali kita berurusan dengan sekumpulan benda yang sejenis, contoh:
  - Lapangan parkir berisi sekumpulan mobil,
  - Sekumpulan barang yang dibeli dari supermarket di dalam kantong belanja,
  - Menu di resto merupakan sekumpulan *tuple* ⟨nama makanan, harga⟩,
  - Daftar semua mahasiswa yang mengambil mata kuliah IF2210,
  - Deretan kursi di ruang bioskop,
  - dll.
- Dalam konteks pemrograman (khususnya OOP) kumpulan tersebut disebut dengan *collection*.

# Collection Sebagai Objek

- Dalam OOP, collection bukan sekadar struktur data.
- Collection adalah objek yang:
  - memiliki state (sekumpulan elemen),
  - menjaga invariant-nya sendiri,
  - menyediakan operasi tingkat tinggi terhadap elemen tersebut.

# Collection dan Responsibility

- Contoh:
  - Order memiliki banyak Item.
  - Team memiliki banyak Player.
  - Course memiliki banyak Student.
- Pertanyaan desain:
  - Siapa yang bertanggung jawab menambah/menghapus elemen?
  - Siapa yang menjamin tidak ada duplikasi?
  - Siapa yang menjamin kapasitas?

# Jenis-jenis collection (1)

- Setiap jenis collection dibedakan oleh kontrak operasinya: bagaimana elemen disimpan, diakses, dan dijaga konsistensinya. Contoh:
  - **Ordered collection** memiliki suatu keterurutan
    - Misalnya list/array yang keterurutannya ditentukan oleh indeks, e.g, ada elemen ke-0, elemen ke-1, dst
  - **Sorted collection** seperti *ordered collection*, dengan keterurutan yang merupakan hasil suatu kalkulasi, i.e., terurut berdasarkan value
  - **Set** elemennya tidak boleh berulang
  - ...

# Jenis-jenis collection (2)

- ...
- **Dictionary/Map** merupakan sekumpulan *value* yang diakses melalui *key*
  - **Bag**, *collection* yang berisi daftar item dan berapa kali item tersebut muncul, seperti kantong belanja yang (misal) berisi 12 telur, 2 susu, dan 3 kentang → `map<item,int>`
- **String** dapat dipandang sebagai sebuah *ordered collection of chars*
- Terkadang dibutuhkan *collection* yang jumlah elemennya tetap (misal: pemain basket dalam satu tim yang sedang bermain), terkadang jumlah elemen harus fleksibel
- dll.

# Apa yang bisa dilakukan dengan collection? (1)

- **Akses:** kapasitas & jumlah elemen yg terisi (umum), mengakses sebuah elemen dalam *collection* (tergantung jenis *collection*)
- **Enumerasi:**
  - **For each:** melakukan hal yang sama kepada setiap elemen
  - **Select/filter:** menghasilkan sub-collection berdasarkan suatu kriteria (misal: mahasiswa yang IPK-nya  $> 3$ )
    - **Find/detect:** ambil satu saja dari hasil select
  - ...

# Apa yang bisa dilakukan dengan collection? (2)

- ...
- **Collect/map:** menghasilkan *collection* baru dengan jumlah elemen yang sama, setiap elemen yang baru merupakan hasil transformasi/operasi terhadap elemen yang lama (contoh: perkalian skalar dengan vektor adalah sebuah collect/map, di mana  $3 * [1, 2, 3] = [3*1, 3*2, 3*3]$ )
- **Reduce, fold:** menghasilkan satu nilai dari semua elemen (misal: sum, average, max)



# Apa yang bisa dilakukan dengan *collection*? (3)

- **Testing:** memeriksa apakah elemen-elemen *collection* memenuhi suatu kondisi tertentu
  - Is empty, is full
  - Any: apakah **ada** elemen yang memenuhi suatu kondisi (eq. hasil filter tidak kosong)
  - All: apakah **semua** elemen memenuhi suatu kondisi (eq. hasil filter sama dengan koleksi aslinya)

# Apa yang bisa dilakukan dengan collection? (4)

- **Menambah & mengurangi elemen:**
  - Menambah & mengurangi 1 elemen
  - Menambah semua elemen dari sebuah *collection* ke *collection* lain
  - Mengurangi elemen yang memenuhi kriteria
  - *Collection* dapat memiliki batasan spesifik dalam penambahan & pengurangan elemen, e.g., Queue dapat dipandang sebagai *collection* (sekumpulan objek) yang hanya bisa ditambah di belakang dan dikurangi di depan

# Abstraksi Tingkat Tinggi

- Operasi seperti map, filter, dan reduce memungkinkan kita bekerja pada level:  
“Apa yang ingin dilakukan terhadap sekumpulan elemen?”
- bukan:  
“Bagaimana cara mengiterasi elemen satu per satu?”

# Collection dalam OOP (1)

- Bahasa OO biasanya menyediakan kelas-kelas untuk menangani *collection*, baik *built-in* maupun dalam bentuk *library* tambahan
  - Java: Collection API, Collection Framework (JCF), Stream API,
  - C++: STL,
  - dst.
- Dalam pemrograman prosedural, aksi terhadap *collection* (seperti pada slide sebelumnya) dilakukan dengan iterasi/*loop* terhadap semua elemen
  - jika dilakukan di OOP, itu artinya meng-expose isi dan cara kerja internal *collection* → menyalahi enkapsulasi.

# Collection dalam OOP (2)

- Pada OOP, *collection* adalah objek juga, bisa dikirim *message*. (Contoh pada slide berikutnya.)
- Iterasi/*loop* seharusnya hanya boleh terjadi **di dalam** kelas-kelas *collection*; kelas yang memanfaatkan *collection* **hanya menggunakan** kontrak yang di-expose kelas-kelas *collection*
  - Beberapa bahasa menyediakan dukungan yang lebih atau kurang eksplisit untuk operasi-operasi *collection* sebagai *message*.

# Contoh (1) dalam *Java-like syntax*

- **For-each:** cetak semua isi array:
  - Prosedural:

```
for (i=0; i<10; i++)  
    print(array[i]);
```
  - OOP:

```
array.forEach(elmt -> print(elmt));
```

## Contoh (2) dalam *Java-like syntax*

- **Select/filter:** isi arr2 dengan elemen arr1 yang genap saja:

- Prosedural:

```
j = 0;
for (i=0; i<10; i++)
    if (arr1[i]%2 == 0)
        arr2[j++] = arr1[i];
```

- OOP:

```
arr2 = arr1.select(elmt -> elmt%2 == 0);
```

Perhatikan dengan OO untuk banyak keperluan jadi tidak perlu peduli jumlah elemen.

## Contoh (3) dalam *Java-like syntax*

- **Collect/map:** perkalian skalar dengan vektor,  $\mathbf{v}_2 = 3 \cdot \mathbf{v}_1$ :
  - Prosedural:

```
for (i=0; i<3; i++)  
    v2[i] = 3 * v1[i];
```
  - OOP:

```
v2 = v1.map(component -> 3*component);
```



## Contoh (4) dalam *Java-like syntax*

- **Testing:** “Boleh masuk jika ada anggota rombongan yang dewasa”:
  - Prosedural:

```
bolehMasuk = false;
for (i=0; i<10; i++)
    if (rombongan[i].usia > 17) {
        bolehMasuk = true;
        break;
    }
```
  - OOP:

```
bolehMasuk = rombongan.any(anggota -> anggota.usia > 17);
```

# Takeaways

- Banyak objek dalam sistem secara alami bersifat sebagai *collection*.
- *Collection* adalah objek yang memiliki dan mengelola sekumpulan elemen.
- Operasi terhadap *collection* sebaiknya diekspresikan sebagai *message* tingkat tinggi.
- Klien tidak perlu mengetahui representasi internal atau jumlah elemen secara eksplisit.
- Abstraksi ini meningkatkan keterbacaan dan menjaga enkapsulasi.